Arguments to main() Function

In C programming, the main function is the entry point of a program. When main is called, it can be called without arguments, or even, with arguments. It can take arguments that provide input from the command line. These arguments allow the user to pass data to the program at runtime.

Syntax :

It has two parameters: argc (argument count) and argv (argument vector). These arguments are usually written as:

```
int main(int argc , char *argv[ ])
```

Here, argc (argument count) is an integer that represents the number of command line arguments passed to the program. argv (argument vector) is an array of character pointers that store the actual arguments.

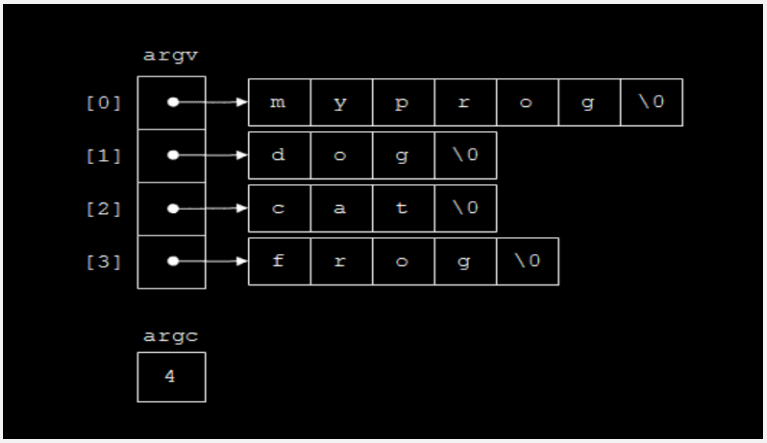
argc (argument count) :

It includes the total number of command line arguments. The count includes the name of the program as the first argument. If no additional arguments are provided, only the program name is counted and argc is 1. When other arguments are passed, the argc counter increases.

argv (argument vector) :

It is an array of strings where each element holds a command line argument. The zeroth index of this array, argv[0], always contains the name of the executable file . Then sequentially other strings argv[1], argv[2], argv[3], ... etc, contain other command line arguments. In this case, arguments are always passed as strings. In case of numbers, they are converted later in the program.

Visual Representation of argc and argv[ ] :



Example Program :

```c
#include<stdio.h>
int main(int argc, char *argv[])
{
    for(int i=0; i<argc; i++)
    {
        printf("argument %d is: %s \n",i,argv[i]);
    }
    return 0;
}
```

Execution and Output :

```
Microsoft Windows [Version 10.0.26100.3476]
(c) Microsoft Corporation. All rights reserved.

C:\Users\230041135>cd C:\Users\230041135\Desktop\assignment4

C:\Users\230041135\Desktop\assignment4>new.exe dhaka monkey12 d0R@emoN 34.65
argument 0 is: new.exe
argument 1 is: dhaka
argument 2 is: monkey12
argument 3 is: d0R@emoN
argument 4 is: 34.65

C:\Users\230041135\Desktop\assignment4>
```

Explanation of the example :

Here 4 arguments have been passed by the user. But the output has 5 arguments including the name of the executable file of the program in the zeroth index of the vector. The program stores the strings inputted in the command line in the argv[ ] which is the array of strings.

Then in the main function a for loop has been iterated until and less than argc, which is the counter of arguments, to access the index of the string array of argv[ ].

Custom Designed Problem :

Write a C program that functions as a simple calculator using command line arguments passing into the main function.

- ➔ Accept three arguments, first and third arguments are numbers and the second argument is the operator.
- ➔ If there are more of less than three arguments passed by the user, display an error message.
- ➔ The numbers must be integers, display an error message if the arguments are anything else but integers, negative integers are also allowed.
- ➔ The calculator can only do (+ , - , x , / , %) operations, display an error message if the operator is anything else but these.
- ➔ For division operation, display an error message if the divisor is zero.

Problem Solution :

```c
#include<stdio.h>
#include<stdlib.h>
#include<ctype.h>
int checkInt(char *str)
{
    int i=0; if(str[0]=='-') i++;
    for(i; str[i]!='\0'; i++) if(!isdigit(str[i])) return 0;
    return 1;
}
int main(int argc, char *argv[])
{
    if(argc!=4) printf("Error: Enter 03 Arguments!\n");
    else if((checkInt(argv[1])!=1)||(checkInt(argv[3])!=1))
        printf("Error: Enter Valid Integers!\n");
    else
    {
        char opr=argv[2][0];
        int n1=atoi(argv[1]);
        int n2=atoi(argv[3]);
        if(opr=='+') printf("%d + %d = %d\n",n1,n2,n1+n2);
        else if(opr=='-') printf("%d - %d = %d\n",n1,n2,n1-n2);
        else if(opr=='x') printf("%d * %d = %d\n",n1,n2,n1*n2);
        else if(opr=='%') printf("%d %% %d = %d\n",n1,n2,n1%n2);
        else if(opr=='/')
        {
            if(n2==0) printf("Error: Enter Valid Divisor!\n");
            else printf("%d / %d = %.2f\n",n1,n2,(float)n1/(float)n2);
        }
        else printf("Error: Enter Valid Operator!\n");
    }
    return 0;
}
```

[NB : asterisk(*) symbol is used as the wildcard symbol, so here ‘x’ is used as the multiplication sign to get input from the command line.]